



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт кибербезопасности и цифровых технологий

Кафедра КБ-4 «Интеллектуальные системы информационной безопасности»

Лабораторная работа №2

Тема: «Анализ логов»

по дисциплине «Технологии обеспечения информационной безопасности»

Выполнил студент группы
ББМО-01-25

Мухаметшин Александр Ринатович

Проверил

Здольник Владимир Вячеславович

Москва 2025

Задание

1. Описание структуры полей логов и разработка парсера: Изучите структуру полей лог-файлов, определите, какие информационные элементы содержатся в каждом лог-сообщении. Разработайте парсер для не менее чем 5 видов сообщений, чтобы выделить каждый элемент информации в отдельные поля.
2. Парсинг лога и сохранение в СУБД: Распарсите лог-файл с использованием разработанного парсера. Сохраните каждую запись в соответствующие таблицы вашей базы данных.
3. Нормализация логов и сохранение в СУБД: Выполните нормализацию логов (приведение к единой форме). Сохраните нормализованные записи в соответствующие таблицы вашей СУБД.
4. Статистический анализ и визуализация: - Проведите статистический анализ данных по различным полям (например, по уровню логирования, частоте определенных событий). - Визуализируйте результаты с использованием библиотек Python, таких как Pandas и Matplotlib.

```
import re
import pandas as pd
import matplotlib.pyplot as plt
import sqlite3
from datetime import datetime
import os
```

Структура строки лога ||| Пример: 20171223-22:15:29:606|Step_LSC|30002312|onStandStepChanged 3579

Основные типы сообщений

1. onStandStepChanged
2. onExtend:
3. **getTodayTotalDetailSteps =**
4. **setTodayTotalDetailSteps=**
5. **REPORT :**
6. **onReceive action:**
7. **processHandleBroadcastAction action:**
8. **calculateCaloriesWithCache totalCalories=**
9. **calculateAltitudeWithCache totalAltitude=**

```
# Регулярные выражения
patterns = {
    'base': re.compile(r'^(\d{8}-\d{2}:\d{2}:\d{2})|([^\|]+\|(\d+)\|(.*)$)'),
    'step': re.compile(r'onStandStepChanged\s+(\d+)'),
    'extend': re.compile(r'onExtend:\s*(\d+)\s+(\d+)\s+(\d+)\s+(\d+)'),
    'get_steps': re.compile(r'getTodayTotalDetailSteps\s*=\s*(\d+)'),
    'set_steps': re.compile(r'setTodayTotalDetailSteps\s*=(\d+)'),
    'report': re.compile(r'REPORT\s*:\s*(\d+)\s+(\d+)\s+(\d+)\s+(\d+)'),
    'screen': re.compile(r'onReceive action:\s*(\d+)'),
    'broadcast': re.compile(r'processHandleBroadcastAction action:(\d+)'),
    'calories': re.compile(r'calculateCaloriesWithCache totalCalories=(\d+)'),
    'altitude': re.compile(r'calculateAltitudeWithCache totalAltitude=(\d+)'),
    'flush': re.compile(r'flush sensor data'),
    'screen_status': re.compile(r'isScreenOn (true|false)'),
    'sync': re.compile(r'startSync|checkCurrentDay|needAutoSync|initDataPrivacy')
}
```

```
def parse_log_line(line):
    line = line.strip()
    if not line:
        return None

    base = patterns['base'].match(line)
    if not base:
        return None

    ts_str, comp, pid, msg = base.groups()

    try:
        timestamp = datetime.strptime(ts_str, '%Y%m%d-%H:%M:%S.%f')
    except:
        return None

    result = {
        'timestamp': timestamp,
        'component': comp.strip(),
```

```
'pid': int(pid),
'message': msg.strip(),
'type': 'unknown'
}

# Определяем тип события
if 'onStandStepChanged' in msg:
    m = patterns['step'].search(msg)
    if m:
        result['type'] = 'step'
        result['steps'] = int(m.group(1))

elif 'onExtend:' in msg:
    m = patterns['extend'].search(msg)
    if m:
        result['type'] = 'extend'
        result['ext_ts'] = int(m.group(1))
        result['ext_a'] = int(m.group(2))
        result['ext_b'] = int(m.group(3))
        result['ext_c'] = int(m.group(4))

elif 'getTodayTotalDetailSteps' in msg:
    m = patterns['get_steps'].search(msg)
    if m:
        result['type'] = 'get_steps'
        result['detail'] = m.group(1)

elif 'setTodayTotalDetailSteps' in msg:
    m = patterns['set_steps'].search(msg)
    if m:
        result['type'] = 'set_steps'
        result['detail'] = m.group(1)

elif 'REPORT :' in msg:
    m = patterns['report'].search(msg)
    if m:
        result['type'] = 'report'
        result['r_steps'] = int(m.group(1))
        result['r_a'] = int(m.group(2))
        result['r_b'] = int(m.group(3))
        result['r_c'] = int(m.group(4))

elif 'onReceive action:' in msg:
    m = patterns['screen'].search(msg)
    if m:
        result['type'] = 'screen'
        result['action'] = m.group(1).strip()

elif 'processHandleBroadcastAction action:' in msg:
    m = patterns['broadcast'].search(msg)
    if m:
        result['type'] = 'broadcast'
        result['action'] = m.group(1).strip()

elif 'calculateCaloriesWithCache' in msg:
    m = patterns['calories'].search(msg)
    if m:
        result['type'] = 'calories'
        result['calories'] = int(m.group(1))

elif 'calculateAltitudeWithCache' in msg:
    m = patterns['altitude'].search(msg)
    if m:
        result['type'] = 'altitude'
        result['altitude'] = int(m.group(1))

elif 'flush sensor data' in msg:
    result['type'] = 'flush'

elif 'isScreenOn' in msg:
    m = patterns['screen_status'].search(msg)
    if m:
        result['type'] = 'screen_status'
        result['screen_on'] = m.group(1) == 'true'

elif any(k in msg for k in ['startSync', 'checkCurrentDay', 'needAutoSync', 'initDataPrivacy']):
    result['type'] = 'sync'

return result
```

Основная функция парсинга одной строки лога в структурированный словарь. Вход: line — строка из файла лога (например: 20171223-22:15:29:606|Step_LSC|30002312|onStandStepChanged 3579)

Выход: Словарь с полями:

timestamp — datetime component — строка pid — целое число message — исходное сообщение type — тип события (step, report, screen, и т.д.) дополнительные поля в зависимости от типа

```
#2. Чтение файла и парсинг

# Путь к файлу
LOG_FILE = 'HealthApp_2k.log'

if not os.path.exists(LOG_FILE):
    raise FileNotFoundError(f"Файл {LOG_FILE} не найден.")

parsed_logs = []

with open(LOG_FILE, 'r', encoding='utf-8') as f:
    for line_num, line in enumerate(f, 1):
        try:
            parsed = parse_log_line(line)
            if parsed:
                parsed_logs.append(parsed)
        except Exception as e:
            print(f"Ошибка парсинга строки {line_num}: {e}")
            continue

print(f"Успешно распаршено строк: {len(parsed_logs)}")

df = pd.DataFrame(parsed_logs)
df.head()
```

Успешно распаршено строк: 1368

	timestamp	component	pid	message	type	steps	ext_ts	ext_a	ext_b
0	2017-12-23 22:15:29.606	Step_LSC	30002312	onStandStepChanged 3579	step	3579.0	NaN	NaN	NaN
1	2017-12-23 22:15:29.615	Step_LSC	30002312	onExtend:1514038530000 14 0 4	extend	NaN	1.514039e+12	14.0	0.0
2	2017-12-23 22:15:29.633	Step_StandReportReceiver	30002312	onReceive action: android.intent.action.SCREEN_ON	screen	NaN	NaN	NaN	NaN
3	2017-12-23 22:15:29.635	Step_LSC	30002312	processHandleBroadcastAction action:android.in...	broadcast	NaN	NaN	NaN	NaN
4	2017-12-23 22:15:29.635	Step_StandStepCounter	30002312	flush sensor data	flush	NaN	NaN	NaN	NaN

#3. Сохранение в SQLite (нормализованная форма)

```
conn = sqlite3.connect('healthapp_logs.db')
cursor = conn.cursor()

# Основная таблица событий
cursor.execute('''
CREATE TABLE IF NOT EXISTS events (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp TEXT,
    component TEXT,
    pid INTEGER,
    event_type TEXT,
    message TEXT
)
''')

# Специализированные таблицы
cursor.execute('CREATE TABLE IF NOT EXISTS step_events (event_id INTEGER, steps INTEGER, FOREIGN KEY(event_id) REFERENCES events(id))')
cursor.execute('CREATE TABLE IF NOT EXISTS extend_events (event_id INTEGER, ext_ts INTEGER, a INTEGER, b INTEGER, c INTEGER, FOREIGN KEY(event_id) REFERENCES events(id))')
cursor.execute('CREATE TABLE IF NOT EXISTS report_events (event_id INTEGER, steps INTEGER, a INTEGER, b INTEGER, c INTEGER, FOREIGN KEY(event_id) REFERENCES events(id))')
cursor.execute('CREATE TABLE IF NOT EXISTS screen_events (event_id INTEGER, action TEXT, FOREIGN KEY(event_id) REFERENCES events(id))')
cursor.execute('CREATE TABLE IF NOT EXISTS calories_events (event_id INTEGER, calories INTEGER, FOREIGN KEY(event_id) REFERENCES events(id))')

for _, row in df.iterrows():
    cursor.execute('''
INSERT INTO events (timestamp, component, pid, event_type, message)
VALUES (?, ?, ?, ?, ?)
''', (row['timestamp'].isoformat(), row['component'], row['pid'], row['type'], row['message']))
    event_id = cursor.lastrowid
```

```

if row['type'] == 'step' and pd.notna(row['steps']):
    cursor.execute('INSERT INTO step_events (event_id, steps) VALUES (?, ?)', (event_id, row['steps']))
elif row['type'] == 'extend' and pd.notna(row['ext_ts']):
    cursor.execute('INSERT INTO extend_events (event_id, ext_ts, a, b, c) VALUES (?, ?, ?, ?, ?)',
                  (event_id, row['ext_ts'], row['ext_a'], row['ext_b'], row['ext_c']))
elif row['type'] == 'report' and pd.notna(row['r_steps']):
    cursor.execute('INSERT INTO report_events (event_id, steps, a, b, c) VALUES (?, ?, ?, ?, ?)',
                  (event_id, row['r_steps'], row['r_a'], row['r_b'], row['r_c']))
elif row['type'] in ['screen', 'broadcast']:
    action = row.get('action')
    if action:
        cursor.execute('INSERT INTO screen_events (event_id, action) VALUES (?, ?)', (event_id, action))
elif row['type'] == 'calories' and pd.notna(row['calories']):
    cursor.execute('INSERT INTO calories_events (event_id, calories) VALUES (?, ?)', (event_id, row['calories']))

conn.commit()
print("Данные сохранены в БД")

```

Данные сохранены в БД

Для каждой строки df:

Вставляется запись в events.

Получается event_id = cursor.lastrowid.

В зависимости от type — вставка в соответствующую дочернюю таблицу.

Нормализация:

нет повторяющихся групп, атомарные значения.

нет частичной зависимости (все поля зависят от id).

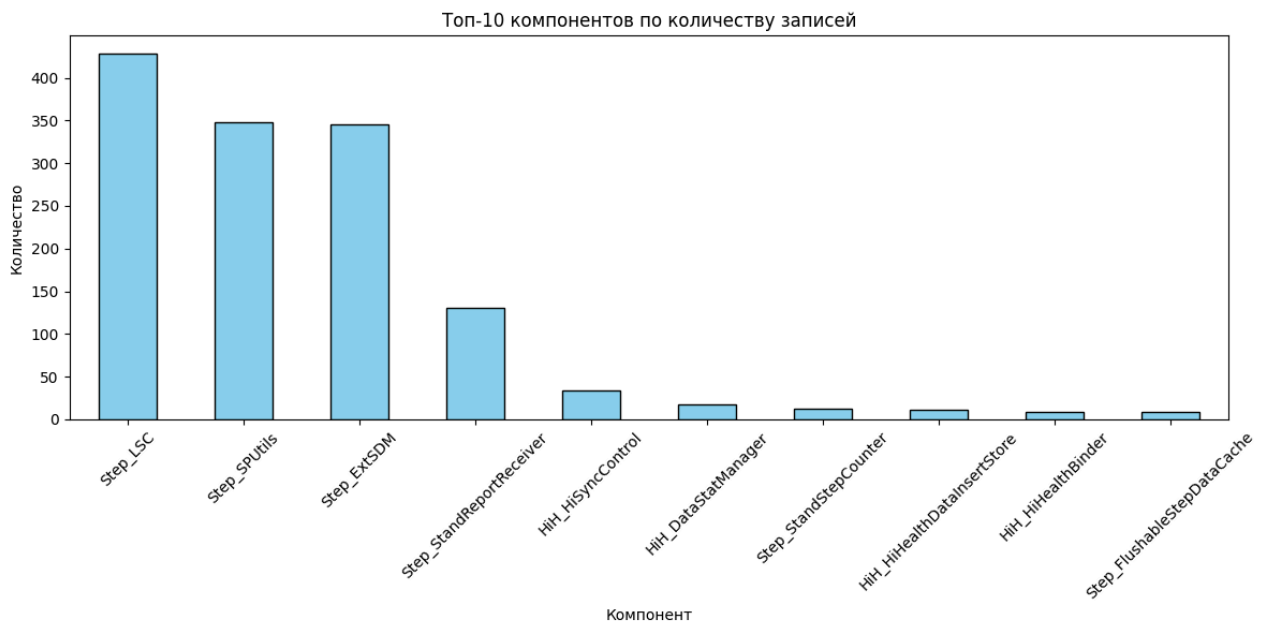
нет транзитивной зависимости.)

#4. Статистический анализ и визуализация

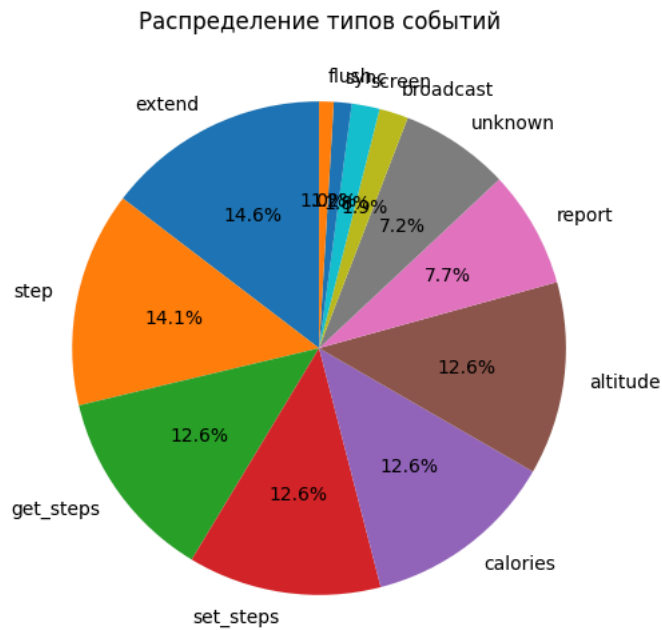
```

# 1. Распределение по компонентам
plt.figure(figsize=(12, 6))
comp_counts = df['component'].value_counts().head(10)
comp_counts.plot(kind='bar', color='skyblue', edgecolor='black')
plt.title('Топ-10 компонентов по количеству записей')
plt.xlabel('Компонент')
plt.ylabel('Количество')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```

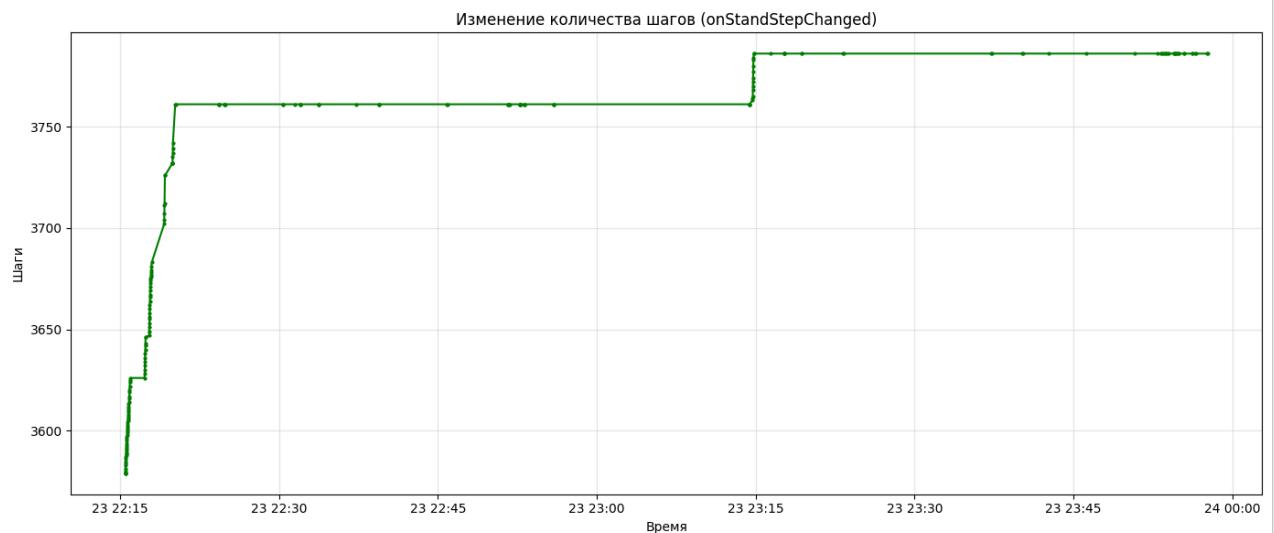


```
# 2. Типы событий
plt.figure(figsize=(10, 6))
type_counts = df['type'].value_counts()
type_counts.plot(kind='pie', autopct='%1.1f%%', startangle=90)
plt.title('Распределение типов событий')
plt.ylabel('')
plt.show()
```



```
# 3. Динамика шагов
step_df = df[df['type'] == 'step'].copy()
step_df = step_df.sort_values('timestamp')

plt.figure(figsize=(14, 6))
plt.plot(step_df['timestamp'], step_df['steps'], 'g.-', markersize=4)
plt.title('Изменение количества шагов (onStandStepChanged)')
plt.xlabel('Время')
plt.ylabel('Шаги')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

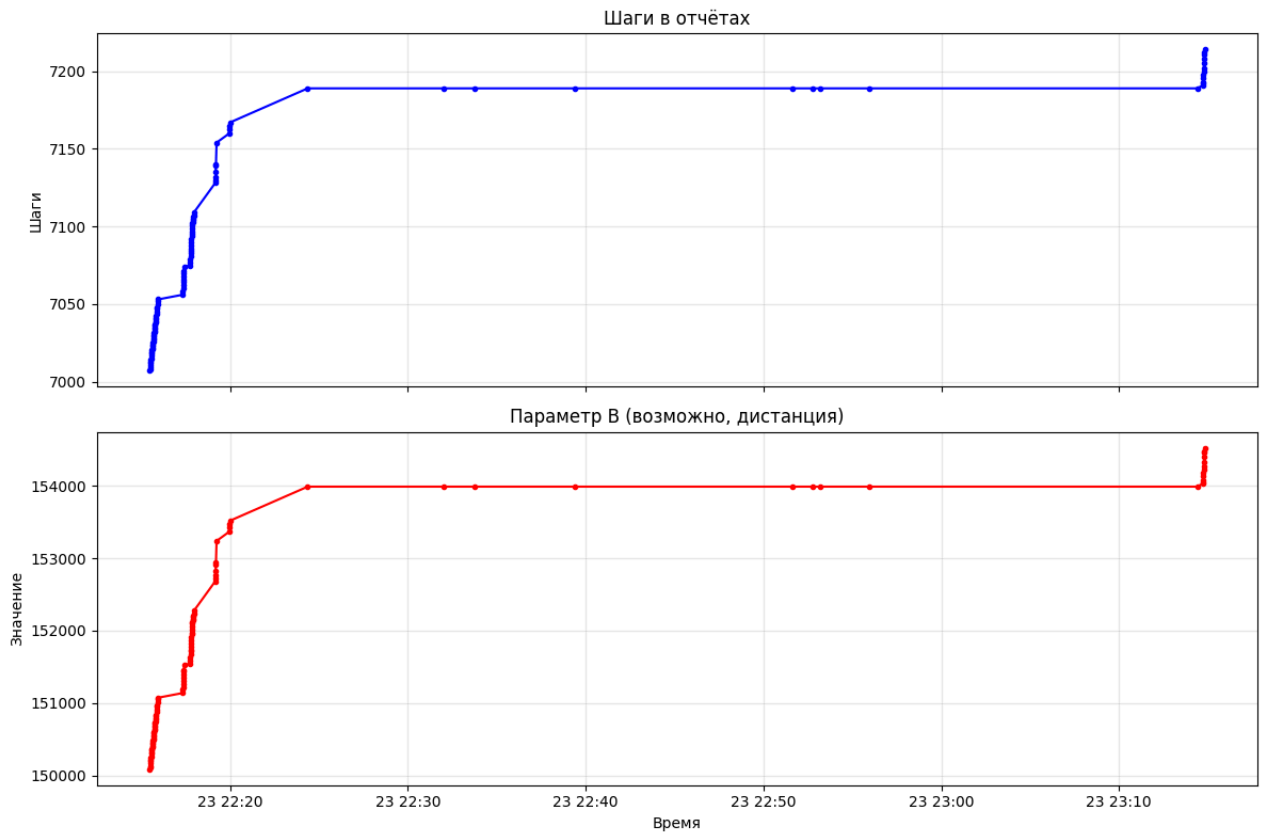


```
# 4. Отчёты
report_df = df[df['type'] == 'report'].copy()
```

```
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8), sharex=True)
ax1.plot(report_df['timestamp'], report_df['r_steps'], 'b-o', markersize=3)
ax1.set_title('Шаги в отчётах')
ax1.set_ylabel('Шаги')
ax1.grid(True, alpha=0.3)

ax2.plot(report_df['timestamp'], report_df['r_b'], 'r-o', markersize=3)
ax2.set_title('Параметр В (возможно, дистанция)')
ax2.set_ylabel('Значение')
ax2.set_xlabel('Время')
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



```
# 5. Экран: включения/выключения
screen_df = df[df['type'].isin(['screen', 'broadcast'])].copy()
screen_df['action_simple'] = screen_df['action'].apply(
    lambda x: 'ON' if 'SCREEN_ON' in x else 'OFF' if 'SCREEN_OFF' in x else 'OTHER'
)

plt.figure(figsize=(10, 4))
screen_df['action_simple'].value_counts().plot(kind='bar', color=['lightgreen', 'lightcoral', 'lightgray'])
plt.title('События экрана')
plt.xlabel('Действие')
plt.ylabel('Количество')
plt.show()
```



#5. Итоговая статистика

```
print(f"Период: {df['timestamp'].min()} – {df['timestamp'].max()}")
print(f"Всего записей: {len(df):,}")
print(f"Уникальных компонентов: {df['component'].nunique()}")
print(f"Типы событий: {'', '.join(sorted(df['type'].unique()))}")
```

```
print(f"\nТоп-5 компонентов:")
for comp, count in df['component'].value_counts().head().items():
    print(f"  {comp}: {count:,}")
```

```
step_stats = df[df['type'] == 'step']['steps']
print(f"\nШаги (onStandStepChanged):")
print(f"  Мин: {step_stats.min():,}, Макс: {step_stats.max():,}, Среднее: {step_stats.mean():.1f}")
```

```
report_steps = df[df['type'] == 'report']['r_steps']
print(f"\nОтчёты:")
print(f"  Шаги: от {report_steps.min():,} до {report_steps.max():,}")
print(f"  Всего отчётов: {len(report_steps):,}")
```

```
calories = df[df['type'] == 'calories']['calories']
if not calories.empty:
    print(f"  Калории: среднее {calories.mean():.0f}, всего {len(calories)} записей")
```

Период: 2017-12-23 22:15:29.606000 – 2017-12-23 23:57:42.470000

Всего записей: 1,368

Уникальных компонентов: 15

Типы событий: altitude, broadcast, calories, extend, flush, get_steps, report, screen, set_steps, step, sync, unknown

Топ-5 компонентов:

Step_LSC: 428

Step_SPUtils: 348

Step_ExtSDM: 346

Step_StandReportReceiver: 130

HiH_HiSyncControl: 34

Шаги (onStandStepChanged):

Мин: 3,579.0, Макс: 3,786.0, Среднее: 3707.1

Отчёты:

Шаги: от 7,007.0 до 7,214.0

Всего отчётов: 105

Калории: среднее 129494, всего 173 записей

Вывод

В ходе лабораторной работы по анализу логов была изучена структура логов из набора Loghub (включая Android, Apache, HDFS и другие), разработан Python-скрипт с использованием регулярных выражений для парсинга записей (извлечение timestamp, loglevel, message), а также применены библиотеки Pandas и Matplotlib для подсчета распределения уровней логов и построения визуализации в Jupyter Notebook. Полученные результаты демонстрируют эффективность подхода для обработки больших объемов логов, позволяя выявлять паттерны ошибок и информационных сообщений, что полезно для системного мониторинга и отладки. В целом, работа подтверждает применимость Python-инструментов для автоматизированного анализа логов в реальных сценариях.